Multi-stream Adaptive Offloading of Joint Compressed Video Streams, Feature Streams, and Semantic Streams in Edge Computing Systems

Dieli Hu[1,2,3], *Wen Ji*[1,2,*], *Zhi Wang*[4,2]
[1] Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China
[2] Peng Cheng Laboratory, Shenzhen, China
[3] University of Chinese Academy of Sciences, Beijing, China
[4] Shenzhen International Graduate School, Tsinghua University, Shenzhen, China
E-mail: hudieli21@mails.ucas.ac.cn, jiwen@ict.ac.cn, wangzhi@sz.tsinghua.edu.cn

***Abstract*—Edge computing (EC) is a promising paradigm for serving latency-sensitive video applications. However, massive compressed video transmission and analysis require considerable bandwidth and computing resources, posing enormous challenges for current multimedia frameworks. Novel multi-stream frameworks that incorporate feature streams are more practical. The reason is that feature streams containing compact video frame feature data have a lower bitrate and better serve machine vision tasks. Nevertheless, feature extraction by devices increases the latency and energy consumption of local computing. Therefore, how to offload suitable streams according to video task requirements and system resources is a challenging issue. This paper studies EC-based multi-stream adaptive offloading. We model the multi-stream offloading and computation problem to maximize system utility by jointly optimizing offloading decisions, computation resource allocation, and video frame sampling rates. Frame sampling rates, processing latency, and energy consumption are considered in system utility modeling. The formulated optimization problem is a mixed-integer programming (MIP) problem. We propose an efficient algorithm to address this MIP problem. The proposed algorithm relies on the Hungarian algorithm and improved greedy Markov approximation. The simulation results validate our proposed algorithm's superior performance.**

***Index Terms*—Compressed video streams, feature streams, multi-stream offloading, resource allocation.**

I. Introduction

Video increasingly dominates traffic in mobile networks. By 2028, Ericsson forecasts that video traffic will account for 80 percent of all global mobile data traffic [1]. The rise of video applications requires lower latency, greater bandwidth [2], and more intensive computing. Edge computing (EC) plays a vital role in low-latency services for video streaming tasks [3], [4]. However, the transmission and analysis of large-scale high-bitrate compressed video streams require high bandwidth and large-volume edge computing resources [5]. Thus, the current multimedia frameworks cannot meet the needs of large-scale video analysis due to limitations in computing, communication, and storage resources [6].

*Corresponding author: Wen Ji. This work is supported by the National Key RD Program of China (2022YFF0902403, 2022YFE0125400), the National Natural Science Foundation of China (62072440), the Beijing Natural Science Foundation (L221004), and the Shenzhen Science and Technology Program (Grant No. JCYJ20220818101014030).

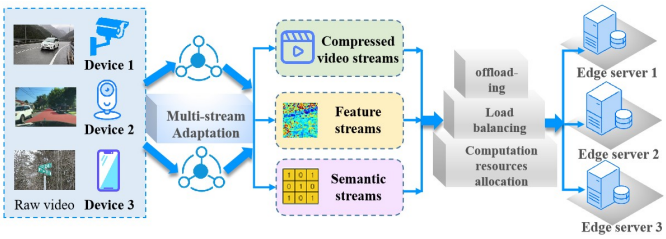


Fig. 1. System model for multi-stream adaptive offloading

Novel multi-stream frameworks were proposed for multimedia systems [5] and smart cities [6], [7], replacing conventional compressed video streams with hybrid compressed video, feature, and semantic streams. In these frameworks, the compressed video stream mainly serves human vision, the feature stream containing compact video frame feature data serves machine vision [6], [7], and the semantic stream is semantic and information-aware notifications to facilitate task processing [5]. Such frameworks can support various applications simultaneously (e.g., target recognition and human-assisted traffic violation monitoring) because both feature and visual data are available [6]. When the compact feature stream is offloaded to the edge server, the bitrate is much lower than in the compressed video [7], thereby steeply reducing bandwidth requirements. Moreover, the data analysis operation is implemented from feature and semantic data, not the total video pixels reconstructed from the compressed video. Computing and cache resources are heavily conserved in this manner [5]. Despite the above advantages of the feature stream, feature extraction on the front-end devices also leads to higher computation latency and energy consumption. Furthermore, the feature stream cannot serve human vision but only machine vision. Therefore, adaptive offloading methods are crucial for multi-stream frameworks based on EC to select the most suitable data stream to offload according to different task requirements, latency constraints, energy consumption, and system resources.

In traditional EC networks, all users must offload their tasks to one EC server [4], [8], [9]. Thus, some tasks may not be accomplished within the required latency. The works in [10], [11] considered multiple EC servers and selected one as its target offload server. However, they did not consider the contention for computation resources between multiple tasks and load balancing scheduling between edge nodes. The authors in [3] indirectly balance computation resources by optimizing configuration selection. Moreover, as the frame sampling rates significantly impact the streaming tasks' service quality, processing latency, and device energy consumption [3], [8], optimizing the frame sampling rates is also necessary.

In this work, we consider a system consisting of multiple front-end devices and edge servers. As shown in Fig.1, the devices can offload the compressed video streams to the edge servers for feature extraction or offload the feature streams to the servers for further video analysis. Moreover, a very low bit-rate semantic stream containing device ID, timestamp, location, and other semantic data based on video preprocessing is required to facilitate task processing [5]. However, this multi-stream adaptive offloading system presents several key challenges that must be addressed. Firstly, offloading multiple data streams is more complex than conventional compressed streams. The different requirements of the video tasks and the heterogeneity of the front-end and edge devices should be considered to determine which data stream to offload. Secondly, besides deciding which data stream to offload, it is also necessary to determine which edge server to offload the task to and the corresponding computation resource allocation. Optimal offloading decisions can balance the load between servers while maximizing system utility. Thirdly, as mentioned above, different frame rates lead to different task service quality, processing latency, and energy consumption. Increasing the frame rate improves task service quality but leads to increased processing latency and device energy consumption. Optimal frame rates optimize the balance between these three critical metrics. Accordingly, the main contributions of this paper are summarised below.

- We study a multi-stream adaptive offloading model of joint compressed video streams, feature streams, and semantic streams based on EC. Multi-stream offloading and computation are modeled to execute video tasks of heterogeneous front-end devices.
- We formulate the multi-stream offloading and computation problem, aiming to maximize system utility by optimizing the adaptive strategy that jointly considers the multi-stream offloading decisions, computation resource allocation at edge servers, and video frame sampling rates. The system utility weighs each device's video frame sampling rate, processing latency, and energy consumption balance.
- The formulated problem is a mixed-integer programming (MIP) and nonconvex problem that is difficult to solve. We propose a low-complexity joint multi-stream adaptive offloading and resource allocation (JMAORA) algorithm to solve it. The JMAORA algorithm utilizes the Hungarian algorithm to obtain the initial solution for offloading decisions and the proposed greedy Markov approximation to optimize the adaptive strategy. The sub-gradient method and Lagrange duality method are leveraged to get the solution for computation resource allocation and frame sampling rates. Finally, the simulation results validate that the JMAORA algorithm achieves superior performance compared to all other baseline schemes.

II. System Model

Suppose there are K front-end devices or video tasks (denoted as $\{v_1, v_2, \cdots, v_K\}$), N multi-CPU core edge servers (denoted as $\{e_1, e_2, \cdots, e_N\}$), and one micro edge server e_0. Moreover, the ID set of these front-end devices is denoted as $\mathcal{K} = \{1, 2, \cdots, K\}$. Assume that time is divided into discrete time slots. In each slot, each device has a video task and can offload a semantic stream and a compressed video stream together to a multi-CPU-core edge server, or offload a semantic stream and a feature video stream together to the micro edge server. Then, the network environment remains near-constant during a (short) time slot. The cloud server can collect feature data from the edge servers and perform further video analysis.

A. Multi-stream Task Value Model

The frame sampling rate of v_k in the current slot is f_k. The index set of multi-stream offloading decisions is denoted as $\mathcal{N} = \{0, 1, \cdots, N\}$. x_k^n represents the offloading decision of v_k. $x_k^0 = 1$ means that v_k chooses to offload the feature stream to the micro edge server. v_k extracts the features of each video frame locally and eventually offloads the corresponding feature stream to e_0. $x_k^n = 1\,(1 \leq n \leq N)$ means offloading the compressed video stream to $e_n\,(1 \leq n \leq N)$ for feature extraction. Since frame sampling rate is an important metric for video streaming task service guarantee [8], a strictly concave utility function $R_k\,(f_k)$ is used to characterize the value of v_k [12].

$$R_k = \left(\alpha_k^{\mathrm{F}} x_k^0 + \alpha_k^{\mathrm{C}} \sum\nolimits_{n=1}^{N} x_k^n\right) \ln\left(1 + f_k\right) \tag{1}$$

where $\alpha_k^{\mathrm{F}}, \alpha_k^{\mathrm{C}} \geq 0$ represents the value of the feature stream and compressed video stream, respectively, representing the priority of the feature stream and compressed video stream for v_k.

B. Processing Latency Model

When v_k offloads the compressed data to e_n, the edge server must decode and compute the received data for feature extraction. Therefore, the processing latency per frame is a function of the transmission latency plus the computing latency. Transmission latency is influenced by data size and upload bandwidth together [13]. For a video frame with resolution r_k of v_k , data size is calculated as $d_k^{\mathrm{C}} = \sigma r_k^2$ (in bits), where σ is a constant [3]. Let B_k^n represent the wireless upload bandwidth (Mbps) between v_k and e_n. Thus, the latency per frame for transmitting the compressed stream

of v_k is $T_k^{n,\text{tran}} = \frac{d_k^{\text{C}}}{B_k^n}, n > 0$. The latency per frame for computing the received compressed data from v_k at e_n is $T_k^{n,\text{comp}} = \frac{d_k^{\text{C}}\xi_k}{c_k^n}, n > 0$, where c_k^n and ξ_k indicate the computation resources (i.e., CPU cycles per second) that e_n allocates to v_k and processing intensity (in CPU cycles per bit) of v_k [10], [14], respectively.

If v_k chooses to offload the feature stream, the latency per frame for local computing of v_k is $T_k^{0,\text{comp}} = \frac{d_k^{\text{C}}\xi_k}{c_k^0}$, where c_k^0 indicates the local computation capacity of v_k. The bitrate of the feature stream is significantly reduced compared to the conventional compressed video stream [5]. Let θ represent the size of the compression ratio of the feature stream relative to the compressed video stream. The data size per frame of the feature stream can now be expressed as $d_k^{\text{F}} = \theta\sigma r_k^2$ (in bits). Similarly, the latency per frame for transmitting the feature stream of v_k is $T_k^{0,\text{tran}} = \frac{d_k^{\text{F}}}{B_k^n}$.

Let s_k denote the bitrate of the semantic stream for v_k in the current slot. Then the transmission latency of the semantic stream is $T_k^{\text{S,tran}} = \frac{s_k}{\sum_{n=0}^{N} x_k^n B_k^n}$, and the total task processing latency of v_k can be expressed as

$$T_k = \sum_{n=0}^{N} x_k^n \left(T_k^{n,\text{tran}} + T_k^{n,\text{comp}}\right) f_k + T_k^{\text{S,tran}} \tag{2}$$

We focus on multi-stream offloading between end devices and edge servers. Data analysis and collection latency in the cloud are negligible here because the cloud server has a powerful computing capacity, and the feature data is much less than the original video.

C. Energy Consumption Model

With the proliferation of video services, there is an increase in energy consumption associated with the growth in computing and data volumes. As a result, we consider the energy efficiency of the end devices, because charging mobile devices can be quite inconvenient. Data transmission and local feature extraction account for most of the device's energy consumption. Let γ_k denote the energy consumption of v_k transmitting per bit task [3]. Thus, the energy consumption per frame for transmitting the compressed stream and feature stream of v_k can be expressed as $E_k^{n,\text{tran}} = \gamma_k d_k^{\text{C}},\ n > 0$ and $E_k^{0,\text{tran}} = \gamma_k d_k^{\text{F}}$, respectively. Similarly, the energy consumption for transmitting the semantic stream is $E_k^{\text{S,tran}} = \gamma_k s_k$.

We use P_k to denote the power of v_k for local computing. Then, the computing energy consumption per frame of v_k is $E_k^{0,\text{comp}} = P_k T_k^{0,\text{comp}}$ [15].

By combining the energy consumption expressions for transmitting and computing, we can obtain the total energy consumption of v_k

$$E_k = \left(\sum_{n=0}^{N} x_k^n E_k^{n,\text{tran}} + x_k^0 E_k^{0,\text{comp}}\right) f_k + E_k^{\text{S,tran}} \tag{3}$$

III. Multi-stream adaptive offloading and resource allocation optimization

A. Problem Formulation

Our goal is to maximize the value of video tasks while minimizing latency and energy costs as much as possible. Let the weight parameters $\psi > 0$ and $\omega > 0$ control the balance between task value, processing latency, and energy consumption. The utility function of v_k is $U_k = R_k - \psi T_k - \omega E_k$. We define the sum of all video task utilities as the system utility, i.e., $\sum_{k=1}^{K} U_k$. Define $\mathbf{x} = \{x_k^n | k \in \mathcal{K}, n \in \mathcal{N}\}$, $\mathbf{f} = \{f_k | k \in \mathcal{K}\}$, and $\mathbf{c} = \{c_k^n | k \in \mathcal{K}, n \in \mathcal{N}\}$, respectively. At the beginning of each slot, we solve an optimization problem to determine the adaptive strategy $\{\mathbf{x}, \mathbf{f}, \mathbf{c}\}$. This optimization model describes the problem formally

$$\mathbf{P1}: \max_{\mathbf{x},\mathbf{f},\mathbf{c}} \sum_{k=1}^{K} U_k \tag{4a}$$

$$\text{s.t. } f_k \le \sum_{n=0}^{N} x_k^n \frac{1}{T_k^{n,\text{comp}}},\ \ \forall k \tag{4b}$$

$$\sum_{n=0}^{N} x_k^n T_k^{n,\text{tran}} + \sum_{n=0}^{N} x_k^n T_k^{n,\text{comp}} \le T_k^{\max},\ \forall k \tag{4c}$$

$$f_k \ge f^{\min},\ \ \forall k \tag{4d}$$

$$0 \le \sum_{k=1}^{K} x_k^n c_k^n \le c_n^{\max},\ \ \forall n > 0 \tag{4e}$$

$$\sum_{n=0}^{N} x_k^n = 1,\ \ \forall k \tag{4f}$$

$$x_k^n \in \{0, 1\},\ \ \forall k, n \tag{4g}$$

Constraint (4b) says that the selected frame sampling rates cannot exceed the number of frames that the devices or servers can process per second. Constraint (4c) imposes the processing latency constraint per frame of compressed and feature streams. $f^{\min}$ in constraint (4d) defines the minimum frame rate required for accuracy. Constraint (4e) says that the sum of the computing resources allocated to all the associated devices cannot exceed the server's total computation resources. Constraints (4f) and (4g) restrict the offload indicator to a binary variable, and each device can only offload compressed data or feature data to one edge server.

P1 is a MIP and nonconvex problem. When there are many devices, an exhaustive search is impossible since the computational complexity of the optimal solution for $\mathbf{x}$ is determined by $(N+1)^K$. Moreover, optimization variables are coupled together in both the objective function and constraints, making this problem more challenging. To solve this problem, we propose the JMAORA algorithm. As shown in Algorithm 1, we leverage the Hungarian algorithm to obtain the initial solution of the offloading decisions $\mathbf{x_0}$ and obtain the solution for $\mathbf{f}$ and $\mathbf{c}$ by the Lagrange duality method and the subgradient method. The proposed greedy Markov approximation algorithm is used to iteratively optimize the adaptive strategy $\{\mathbf{x}, \mathbf{f}, \mathbf{c}\}$.

B. Joint Optimization of Frame Rates and Computation Resource Allocation

Once offloading decisions $\mathbf{x}$ are fixed, the joint optimization of frame rates and computation resource allocation problem from $\mathbf{P1}$ is formulated as follows:

$$\mathbf{P2}: \max_{\mathbf{f},\mathbf{c}} \sum_{k=1}^{K} U_k, \quad \text{s.t. (4b)} - \text{(4e)} \tag{5}$$

Due to the coupling of $\mathbf{f}$ and $\mathbf{c}$ in (5), it is easy to prove that $\mathbf{P2}$ is a nonconvex problem according to the Hessian matrix [16]. Therefore, we use the block coordinate descent (BCD) technique to solve $\mathbf{P2}$ by solving the following two convex subproblems alternately until convergence [17]

$$\mathbf{P3}: \max_{\mathbf{f}} \sum_{k=1}^{K} U_k, \quad \text{s.t. (4b), (4d)}, \tag{6}$$

$$\mathbf{P4}: \max_{\mathbf{c}} \sum_{k=1}^{K} U_k, \quad \text{s.t. (4b), (4c), (4e)} \tag{7}$$

1) Solving $\boldsymbol{P3}$ under given $\boldsymbol{c}$: It is easy to prove that $\mathbf{P3}$ is convex. Thus, utilizing the Lagrange duality method [16], we get the optimal frame rates as follows:

$$f_k^* = \begin{cases} f_k^{\text{thre}}, & \text{if } f_k^{\text{upp}} > f_k^{\text{thre}} > f^{\min} \\ f_k^{\text{upp}}, & \text{if } f_k^{\text{upp}} \le f_k^{\text{thre}} \\ f^{\min}, & \text{if } f^{\min} \ge f_k^{\text{thre}} \end{cases} \tag{8}$$

where $f_k^{\text{thre}} = \left\lfloor \frac{\alpha_k^{\text{F}}}{\psi\left(T_k^{0,\text{comp}}+T_k^{0,\text{tran}}\right)+\omega\left(E_k^{0,\text{comp}}+E_k^{0,\text{tran}}\right)} - \frac{1}{2} \right\rfloor$, $f_k^{\text{upp}} = \left\lfloor \frac{c_k^0}{d_k^{\text{C}}\xi_k} \right\rfloor$ for the set of devices that transmit the feature streams $\mathcal{K}_{\text{F}} = \left\{k \in \mathcal{K} \middle| x_k^0 = 1\right\}$, and $f_k^{\text{thre}} = \left\lfloor \frac{\alpha_k^{\text{C}}}{\psi\sum_{n=1}^{N} x_k^n\left(T_k^{n,\text{tran}}T_k^{n,\text{comp}}\right)+\omega E_k^{n,\text{tran}}} - \frac{1}{2} \right\rfloor$, $f_k^{\text{upp}} = \left\lfloor \frac{\sum_{n=1}^{N} x_k^n c_k^n}{d_k^{\text{C}}\xi_k} \right\rfloor$ for the set of devices that transmit the compressed streams $\mathcal{K}_{\text{C}} = \left\{k \in \mathcal{K} \middle| \sum_{n=1}^{N} x_k^n = 1\right\}$, respectively.

2) Solving $\boldsymbol{P4}$ under given $\boldsymbol{f}$: We apply the sub-gradient method [13] to solve $\mathbf{P4}$ to obtain the optimal computation resource allocation. The sub-gradient function of U_k with respect to c_k^n is denoted as $\partial U_k = \frac{\partial U_k}{\partial c_k^n}$. Suppose the sub-gradient function of $\sum_{k=1}^{K} U_k$ is g, then $\mathbf{P4}$ is solved iteratively as

$$c^{(j+1)} = c^{(j)} + \kappa_j g^{(j)} \tag{9}$$

where κ_j is the step size of the j-th iteration and g is defined as

$$g = \begin{cases} \partial\left(\sum_{k=1}^{K} U_k\right), & \text{subject to (4b), (4c) and (4e)} \\ -\partial\left(\sum_{k=1}^{K} c_k^n\right), & \text{if } \sum_{k=1}^{K}\left(x_k^n c_k^n\right) > c_n^{\max} \end{cases} \tag{10}$$

where $-\partial\left(\sum_{k=1}^{K} c_k^n\right)$ is used as the obstacle function.

C. Optimization of Adaptive Strategy

According to the analysis in the previous subsection, $\mathbf{P1}$ is easily solved once offloading decisions $\mathbf{x}$ are determined. Due to the binary vector $\mathbf{x}$, the optimal solution cannot be obtained in polynomial time. Hence, we leverage the Hungarian algorithm [18] to find the optimal initial offloading solution $\mathbf{x}_0$. Then, we propose a low-complexity improved Markov approximation called greedy Markov approximation to update $\{\mathbf{x}, \mathbf{f}, \mathbf{c}\}$ iteratively to obtain the near-optimal solution.

1) The Hungarian algorithm-based initial offloading solution: At first, assuming each task has a volume of 1, and

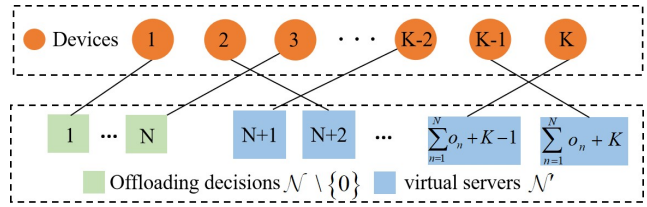


Fig. 2. Bipartite graph for offloading decision matching

e_n has a volume of $o_n = \left\lceil \frac{c_n^{\max}}{\sum_{n=1}^{N} c_n^{\max}} K \right\rceil$ to ensure that the N servers can accommodate K tasks of devices. Then assume that e_n provides an average computation resource $c_k^n = c_n^{\max}/o_n$ to each device connected to it. The purpose of this allocation is to allow servers with more capacity to support more tasks. Since once the offloading decisions $\mathbf{x}$ are fixed, it is equivalent to solving $\mathbf{P3}$. We can easily obtain the optimal frame rate matrix $\mathbf{F}^*$ corresponds to $K*(N+1)$ cases where K devices match $N+1$ offloading decisions. Then, we can construct the optimal utility function matrix associated with the offloading decisions, denoted by $\mathbf{W}^* = (w_{k,n}^*)_{\forall k \in \mathcal{K}, n \in \mathcal{N}}$. Based on the above analysis, the utility function of v_k can be obtained under given $\mathbf{c}$, denoted by $w_k = \sum_{n \in \mathcal{N}} x_{k,0}^n w_{k,n}^*$, where $x_{k,0}^n$ is the initial offloading decision of v_k. Replace x_k^n with $x_{k,0}^n$ in the constraint (4f) and (4g). Thus, this subsection focuses on the optimization of the initial offloading decision problem under given $\mathbf{c}$ from $\mathbf{P1}$ formulated as follows:

$$\mathbf{P5}: \max_{\mathbf{x_0}} \sum_{k=1}^{K} w_k \tag{11a}$$

$$\text{s.t. } 0 \le \sum_{k=1}^{K} x_{k,0}^n \le o_n, \forall n > 0 \tag{11b}$$

$$\text{(4f), (4g)} \tag{11c}$$

Since e_n can serve o_n devices and multiple devices can choose to compute locally to offload feature streams, $\mathbf{P5}$ may contain one-to-many matching cases and is therefore difficult to solve. Our approach is constructing $o_n - 1$ virtual servers for e_n, and K virtual servers for local feature stream computing. Thus, a total of $\sum_{n=1}^{N}(o_n - 1) + K$ virtual servers are constructed in this system. Moreover, the index set of the virtual servers can be expressed as $\mathcal{N}' = \left\{N+1, N+2, \cdots, \sum_{n=1}^{N} o_n + K\right\}$. Then, the utility function when v_k chooses e_n is replicated $o_n - 1$ times, and the utility function when v_k chooses local computing is replicated K times to obtain the reconstructed optimal utility function matrix $\hat{\mathbf{W}}^* = (\hat{w}_{k,n}^*)_{\forall k \in \mathcal{K}, n \in \{\mathcal{N} \setminus \{0\} \cup \mathcal{N}'\}}$. The reconstructed utility function of v_k is denoted as $\hat{w}_k = \sum_{n \in \{\mathcal{N} \setminus \{0\} \cup \mathcal{N}'\}} x_{k,0}^n \hat{w}_{k,n}^*$. Finally, the index set of offloading decisions $\mathcal{N}$ in (11c) is replaced by $\{\mathcal{N} \setminus \{0\} \cup \mathcal{N}'\}$, and $\mathbf{P5}$ is equivalently transformed into the following bipartite matching problem:

$$\mathbf{P6}: \max_{\mathbf{x_0}} \sum_{k=1}^{K} \hat{w}_k \tag{12a}$$

$$\text{s.t. } 0 \le \sum_{k=1}^{K} x_{k,0}^n \le 1, \forall n \in \{\mathcal{N} \setminus \{0\} \cup \mathcal{N}'\} \tag{12b}$$

$$\text{(11c)}$$

As shown in Fig.2, $\mathbf{P6}$ is a bipartite matching problem, and $\mathbf{x}_0$ can be derived by the Hungarian algorithm [18].

2) Greedy Markov approximation-based iterative optimization : After determining $\mathbf{x_0}$, in each iteration, We randomly select a device and change its offloading decision to the server or local device (for feature extraction to offload the feature stream) with the largest task accommodating capacity while leaving the others unchanged. Let $x_k = \left\{x_k^0, x_k^1, \cdots, x_k^N\right\}$ denote the offloading decision vector of

v_k. Suppose $c_n = \left(\left\lfloor \frac{c_n^{\max}}{c_k^{\text{sum}}}K+\frac{1}{2}\right\rfloor - \sum_{k=1}^{K} x_k^n\right) \Big/ \left\lfloor \frac{c_n^{\max}}{c_k^{\text{sum}}}K+\frac{1}{2}\right\rfloor$ is the task accommodating capacity of e_n, and $c_k = \left(\left\lfloor \frac{\delta c_k^0}{c_k^{\text{sum}}}K+\frac{1}{2}\right\rfloor - x_k^0\right) \Big/ \left\lfloor \frac{\delta c_k^0}{c_k^{\text{sum}}}K+\frac{1}{2}\right\rfloor$ is the task accommodating capacity of v_k, where $c_k^{\text{sum}} = \sum_{n=1}^{N} c_n^{\max} + \delta c_k^0$ (the details of the task accommodating capacity are provided in the supplementary material). Here, δ controls the trade-off between the devices' local computation capacity and the servers'. This step mainly utilizes the idea of the greedy algorithm, i.e., always selecting the one with the largest task accommodating capacity. Then, we can use the new offloading decisions $\tilde{\mathbf{x}}$ to solve $\mathbf{P2}$ to obtain the $\tilde{\mathbf{f}}$ and $\tilde{\mathbf{c}}$. The new utility function $\tilde{\mathbf{U}}$ can be calculated and subsequently updated to the new adaptive strategy $\left\{\tilde{\mathbf{x}}, \tilde{\mathbf{f}}, \tilde{\mathbf{c}}\right\}$ with probability ρ while keeping unchanged with probability $1-\rho$, where $\rho = \frac{1}{1+e^{\frac{\mathbf{U}-\tilde{\mathbf{U}}}{\tau}}}$ [3]. The parameter $\tau > 0$ controls the randomness. When the new adaptive strategy $\left\{\tilde{\mathbf{x}}, \tilde{\mathbf{f}}, \tilde{\mathbf{c}}\right\}$ leads to a higher utility function, the adaptive strategy is more likely to be updated because the probability ρ depends on the utility function difference $\mathbf{U}-\tilde{\mathbf{U}}$. The algorithm stops iterating when the upper limit of iterations is reached or when $\left|\mathbf{U}-\tilde{\mathbf{U}}\right| < 0.01$ for more than ten iterations.

Algorithm 1: JMAORA for Solving Problem $\mathbf{P1}$

Input: B_k^n, $c_n^{\max}$, c_k^0, P_k, ξ_k, s_k, r_k, $T_k^{\max}$, γ_k, α_k^{C}, α_k^{F}, $f^{\min}$, ψ, ω, τ, and θ, $\forall k, n$.

Obtain $\mathbf{F}^*$ by solving $\mathbf{P3}$ using Eq.(8);

Obtain $\mathbf{x_0}$ by solving $\mathbf{P6}$ using the Hungarian algorithm [18], and set $\mathbf{x} \leftarrow \mathbf{x_0}$;

repeat

  Randomly select a device and change x_k to $\tilde{x}_k$ by selecting the server or local device with the largest task accommodating capacity;

  if *new offloading decisions* $\tilde{\mathbf{x}}$ *are feasible* **then**

    Decouple $\mathbf{P2}$ into $\mathbf{P3}$ and $\mathbf{P4}$ using the BCD technique. Then, obtain $\tilde{\mathbf{f}}$ by solving $\mathbf{P3}$ using Eq.(8) and $\tilde{\mathbf{c}}$ by solving $\mathbf{P4}$ based on Eq.(9);

    With probability ρ update to the new adaptive strategy $\mathbf{x} \leftarrow \tilde{\mathbf{x}}$, $\mathbf{f} \leftarrow \tilde{\mathbf{f}}$, $\mathbf{c} \leftarrow \tilde{\mathbf{c}}$;

until *iteration stop condition is reached*;

Output: $\mathbf{x}$, $\mathbf{f}$ and $\mathbf{c}$.

IV. Simulation Results

In this section, we compare the performance of the proposed JMAORA algorithm with seven baseline schemes. The number of devices and edge servers in this system is set as 40 and 4, respectively. According to [11], we analyze the semantic data size for 15 days dataset and then consider s_k is uniformly distributed in $[400, 720]$ bps. B_k^n, $c_n^{\max}$, c_k^0, ξ_k, r_k, α_k^{F} and $T_k^{\max}$ are uniformly distributed within the intervals $[1, 80]$ Mbps [9], $[10, 15]$ GHz [10], $\{0.5, 0.8, 1.0\}$ GHz, $[2500, 4000]$ cycles/bit [14], $\{360\text{p}, 540\text{p}, 720\text{p}, 900\text{p}, 1080\text{p}\}$, $[0.4, 0.6]$ and $[0.1, 1]$ s, respectively. P_k corresponding to c_k^0 is set as $\{0.5, 0.75, 0.9\}$W. The remaining parameters are set as $\gamma_k = 0.5 \times 10^{-5}$J [3], $\alpha_k^{\text{C}} = 0.5$ $f^{\min} = 2$ fps [8], $\psi = 0.5$, $\omega = 0.5$, and $\tau = 0.1$.

The following seven baseline schemes are used to validate the performance of the proposed JMAORA algorithm:

MAM: This scheme's idea is to update $\{\mathbf{x}, \mathbf{f}, \mathbf{c}\}$ iteratively based on the Markov approximation method (as in [3], [19]), rather than the proposed greedy Markov approximation.

BCD: This scheme's idea is to optimize $\{\mathbf{x}, \mathbf{f}\}$ and $\mathbf{c}$ by the BCD technique, as in [17].

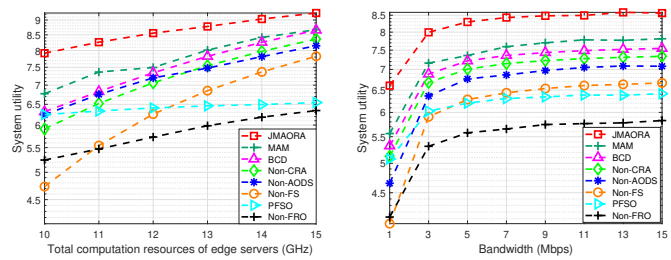
Non-CRA: It is a scheme without computation resource allocation, where computation resources are given.

Non-AODS: It is a scheme without adaptive offloading decision selection, where the decisions are given.

Non-FRO: It is a scheme without frame rate optimization, where frame rates are given.

Non-FS: It is a scheme without feature streams.

PFSO: This scheme prioritizes feature stream offloading as long as v_k satisfies the latency constraint (4c) when $x_k^0 = 1$.

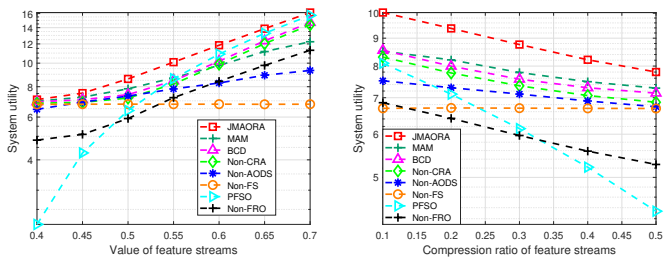


(a) (b)

Fig. 3. Comparison of different schemes with varying (a) computation capacity of edge servers, and (b) bandwidth.

Fig.3(a) and Fig.3(b) show the utility performance of JMAORA and other seven baseline schemes when the computation capacity of edge servers (i.e., $c_n^{\max}, \forall n$) and bandwidth (i.e., $B_k^n, \forall k, n$) vary, respectively. Here, it is assumed that $\theta = 0.3$ (it is shown that feature streams can reduce the bitrate of transmission by an average of 70% compared to compressed video streams in [5]). The bandwidth is uniformly distributed in $[1, 80]$ Mbps in Fig.3(a) and total computation resources are uniformly distributed in $[10, 15]$ GHz in Fig.3(b). As revealed by Fig.3(a) and Fig.3(b), our proposed JMAORA algorithm can outperform the seven baseline schemes in all computation capacity or bandwidth regions. Moreover, it is easy to see that the system utility of JMAORA is significantly higher than that of Non-FS, which confirms the superiority of multi-stream adaptive offloading compared to conventional compressed video streams. Fig.3(a) shows that the system utility increases for all schemes as the computation capacity of servers increases because more computation resource allocation leads to less computing latency, so the system utility increases. More specifically, Fig.3(a) indicates that JMAORA can achieve up to 25.36% more system utility than BCD and 16.94% more system utility than MAM. Thus, it can be concluded that the greedy Markov approximation gives a near-optimal solution for offloading decision selection. This is because the greedy Markov approximation outperforms the Markov approximation method (called MAM), which has been proven to be approximately optimal. As shown in Fig.3(b), the system utility increases for all schemes as the bandwidth increases. However, when the bandwidth size surpasses a point (i.e., $B_k^n \geq 3$ Mbps,$\forall k.n$), the system utility increases

slowly and eventually remains constant. This result reveals a phenomenon where the system utility does not improve once the channel quality exceeds a critical point.



(a) (b)

Fig. 4. Comparison of different schemes with varying (a) value of feature streams, and (b) compression ratio of feature streams.

Fig.4(a) and Fig.4(b) show how the value of feature streams (i.e., $\alpha_k^{\mathrm{F}}, \forall k$) and compression ratio of feature streams (i.e., θ) affect the system utility, respectively. Here, it is assumed that $\theta = 0.3$ in Fig.4(a) and α_k^{F} is uniformly distributed in $[0.4, 0.6]$ in Fig.4(b). Again, it can be seen that our proposed JMAORA algorithm can significantly improve system utility compared with other baseline schemes. Fig.4(a) illustrates the increase in system utility as the value of feature streams increases. Furthermore, it can be found that when $\alpha_k^{\mathrm{F}} < \alpha_k^{\mathrm{C}}$, the system utility of PFSO is much lower than the other schemes, while when $\alpha_k^{\mathrm{F}} > \alpha_k^{\mathrm{C}}$, the system utility of PFSO increases greatly and is closer to JMAORA. Thus, it is possible to adjust the importance of compressed and feature streams based on different vision task requirements. As shown in Fig.4(b), as the compression ratio increases, the system utility decreases for all schemes except Non-FS. This is due to the increase in feature data size, leading to an increase in transmission latency and transmission energy consumption for the feature streams. Moreover, Fig.4(b) confirms again that JMAORA outperforms MAM by a wide margin.

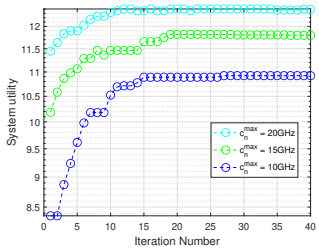


Fig. 5. Convergence of proposed algorithm for different $c_n^{\max}$

Fig.5 displays the convergence of the proposed JMAORA algorithm for $c_n^{\max} = 20$ GHz, $c_n^{\max} = 15$ GHz, $c_n^{\max} = 10$ GHz, $\forall n$. As can be seen in Fig.5, JMAORA always converges to the optimal solution within twenty iterations. Fig.5 also illustrates that the utility achieved is about 90% of its maximum value in fewer than five iterations. Referring back to Fig.3 and Fig.4, JMAORA has been proven to outperform MAM significantly. Therefore, JMAORA is very efficient, despite achieving a locally optimal solution.

V. Conclusion

In this paper, we study a novel EC-based multi-stream adaptive offloading system to support different video tasks while maximizing system utility. An efficient algorithm, JMAORA, was designed to tackle the formulated MIP problem. The superior performance of the proposed algorithm was verified by simulations. In future work, exhaustive experiments will be conducted to analyze the bitrate relationship between the compressed and feature streams. Furthermore, more metrics will be considered by the adaptive strategy, such as frame resolution, video bitrate, etc.

References

[1] Ericsson mobility report 2022, https://www.ericsson.com/en/reports-and-papers/mobility-report/reports/november-2022.

[2] Cisco annual internet report (2018–2023) white paper, https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490. html.

[3] S. Zhang, C. Wang, Y. Jin, et al., "Adaptive configuration selection and bandwidth allocation for edge-based video analytics," *IEEE/ACM Trans. on Networking*, vol. 30, no. 1, pp. 285-298, 2022.

[4] J. Chakareski and S. Gupta, "Multi-connectivity and edge computing for ultra-low latency lifelike virtual reality," in *IEEE ICME*, pp. 1-6, 2020.

[5] W. Ji, B. Liang, Y. Wang, et al., "Crowd V-IoE: Visual Internet of Everything architecture in AI-driven fog computing," *IEEE Wireless Commun*, vol. 27, no. 2, pp. 51-57, 2020.

[6] W. Gao, S. Ma, L. Duan, et al., "Digital retina: A way to make the city brain more efficient by visual coding," *IEEE Trans. Circuits Syst. Video Technol*, vol. 31, no. 11, pp. 4147–4161, 2021.

[7] Y. Lou, L. Duan, Y. Luo, et al., "Towards digital retina in smart cities: A model generation, utilization and communication paradigm," in *IEEE ICME*, pp. 19-24, 2019.

[8] X. Ran, H. Chen, X. Zhu, et al., "DeepDecision: A mobile deep learning framework for edge video analytics," in *IEEE INFOCOM*, pp. 1421–1429, 2018.

[9] P. Yang, F. Lyu, W. Wu, et al., "Edge coordinated query configuration for low-latency and accurate video analytics," *IEEE Trans Ind. Informat*, vol. 16, no. 7, pp. 4855-4864, 2020.

[10] T. Ouyang, X. Chen, Z. Zhou, et al., "Adaptive user-managed service placement for mobile edge computing via contextual multiarmed bandit learning," *IEEE Trans. Mobile Comput*, vol. 22, no. 3, pp. 1313–1326, 2023.

[11] S. Wang, Y. Guo, N. Zhang, et al., "Delay-aware microservice coordination in mobile edge computing: A reinforcement learning approach," *IEEE Trans. Mobile Comput*, vol. 20, no.3, pp. 939-953, 2021.

[12] W. Ji, L. Duan, X. Huang, et al., "Astute video transmission for geographically dispersed devices in visual IoT systems," *IEEE Trans. Mob Comput*, vol. 21, no. 2, pp. 448-464, 2022.

[13] J. Ren, G. Yu, Y. Cai, et al., "Latency optimization for resource allocation in mobile-edge computation offloading," *IEEE Trans. Wireless Commun*, vol. 17, no. 8, pp. 5506–5519, 2018.

[14] J. Kwak, Y. Kim, J. Lee, et al., "Dream: Dynamic resource and task allocation for energy minimization in mobile cloud systems," *IEEE J. Sel. Areas Commun*, vol. 33, no. 12, pp. 2510–2523, 2015.

[15] L. Zeng, X. Chen, Z. Zhou, et al., "Coedge: Cooperative DNN inference with adaptive workload partitioning over heterogeneous edge devices," *IEEE/ACM Trans. Netw*, vol. 29, no. 2, pp. 595–608, 2021.

[16] S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge, U.K.: Cambridge Univ. Press, 2004.

[17] T. Bai, C. Pan, Y. Deng, et al., "Latency minimization for intelligent reflecting surface aided mobile edge computing," *IEEE J. Sel. Areas Commun*, vol. 38, no. 11, pp. 2666–2682, 2020.

[18] C. Guo, L. Liang, and G. Y. Li, "Resource allocation for low-latency vehicular communications: An effective capacity perspective," *IEEE J. Sel. Areas Commun*, vol. 37, no. 4, pp. 905–917, 2019.

[19] T. Ouyang, Z. Zhou, and X. Chen, "Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing," *IEEE J. Sel. Areas Commun*, vol. 36, no. 10, pp. 2333–2345, 2018.